

Design Patterns

Command Pattern

the Command Pattern

✦ ***Encapsulating Invocation*** ✦

In this chapter, we take encapsulation to a whole new level:

we're going to encapsulate method invocation.



Home Automation or Bust, Inc.
1221 Industrial Avenue, Suite 2000
Future City, IL 62914

Greetings!

I recently received a demo and briefing from Johnny Hurricane, CEO of Weather-O-Rama, on their new expandable weather station. I have to say, I was so impressed with the software architecture that I'd like to ask you to design the API for our new Home Automation Remote Control. In return for your services we'd be happy to handsomely reward you with stock options in Home Automation or Bust, Inc.

I'm enclosing a prototype of our ground-breaking remote control for your perusal. The remote control features seven programmable slots (each can be assigned to a different household device) along with corresponding on/off buttons for each. The remote also has a global undo button.

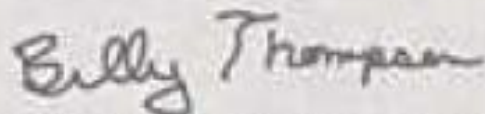
I'm also enclosing a set of Java classes on CD-R that were created by various vendors to control home automation devices such as lights, fans, hot tubs, audio equipment, and other similar controllable appliances.

We'd like you to create an API for programming the remote so that each slot can be assigned to control a device or set of devices. Note that it is important that we be able to control the current devices on the disc, and also any future devices that the vendors may supply.

Given the work you did on the Weather-O-Rama weather station, we know you'll do a great job on our remote control!

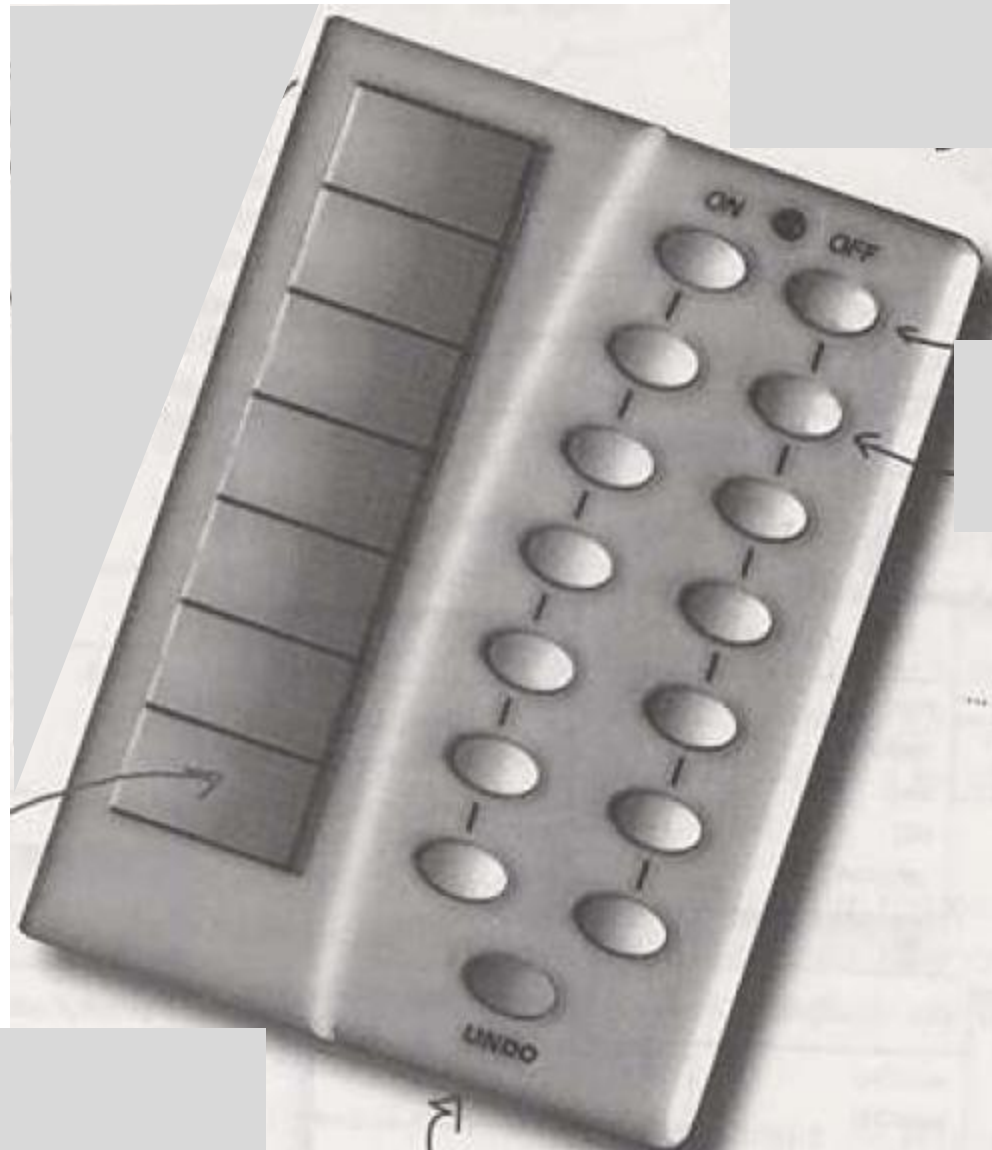
We look forward to seeing your design.

Sincerely,

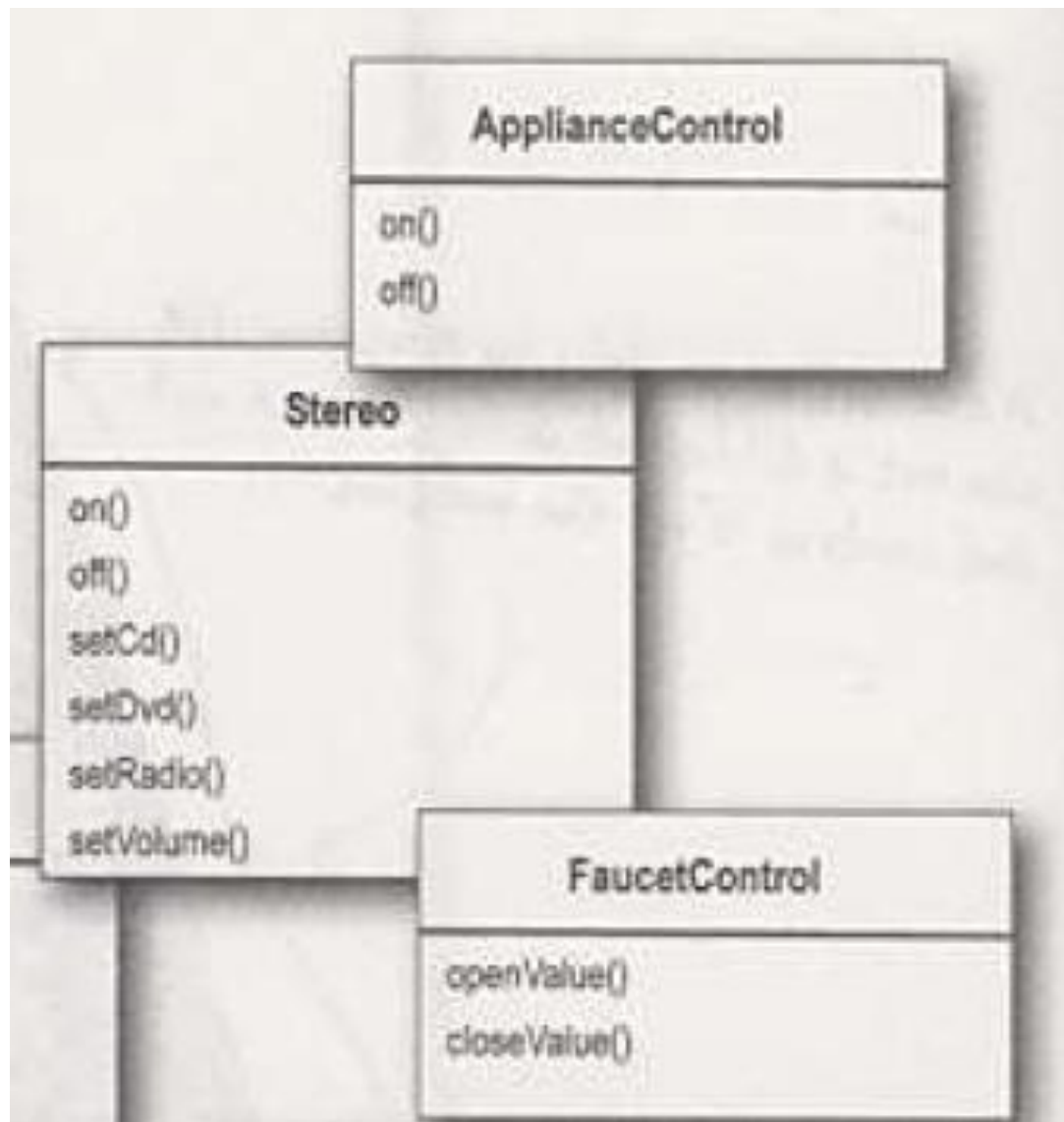
A handwritten signature in cursive script that reads "Billy Thompson".

Bill "X-10" Thompson, CEO

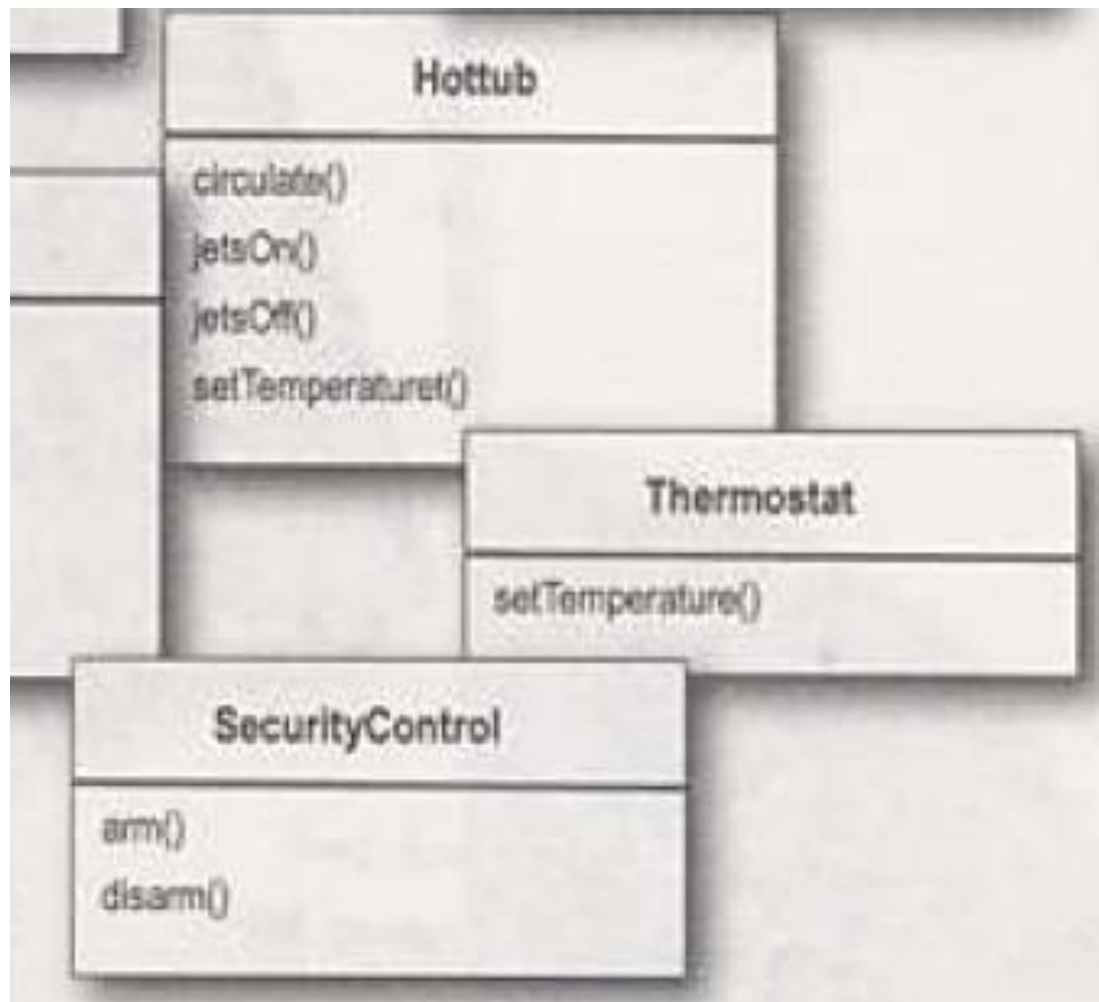
Free hardware! Let's check out the Remote Control...



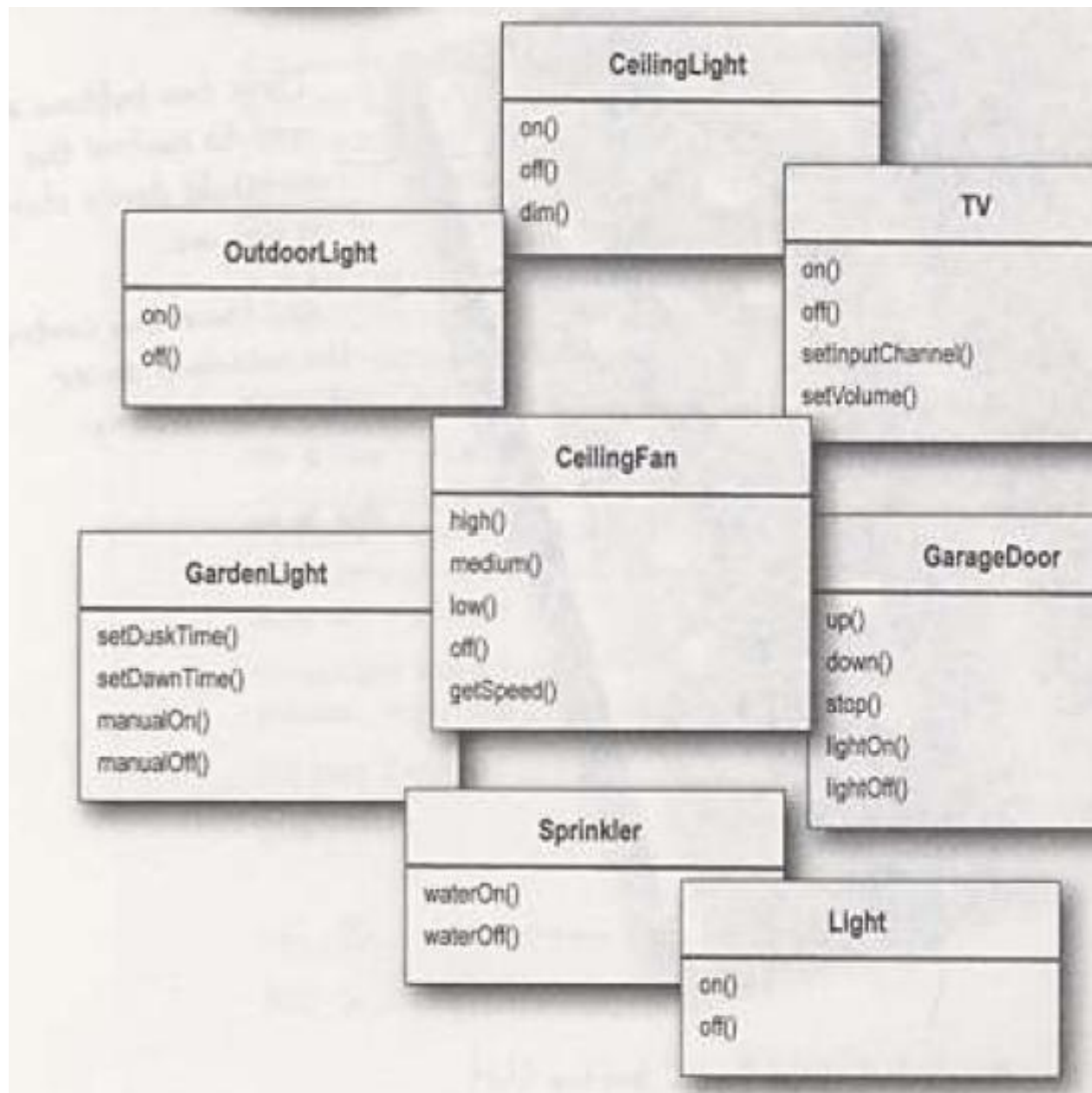
Taking a look at the vendor classes



Taking a look at the vendor classes



Taking a look at the vendor classes



Our first command object

Implementing the Command interface

```
public interface Command {  
    public void execute();  
}
```

Implementing a Command to turn a light on

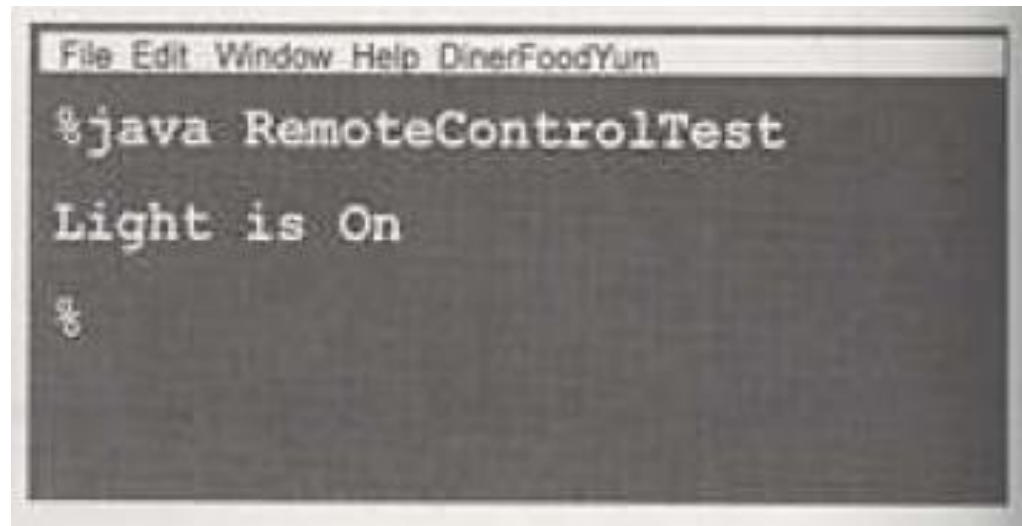
```
public class LightOnCommand implements Command {  
    Light light;  
    public LightOnCommand(Light light) {  
        this.light = light;  
    }  
    public void execute() {  
        light.on();  
    }  
}
```

Using the command object

```
public class SimpleRemoteControl {  
    Command slot;  
    public SimpleRemoteControl() {}  
    public void setCommand(Command command) {  
        slot = command;  
    }  
    public void buttonWasPressed() {  
        slot.execute();  
    }  
}
```

Creating a simple test to use the Remote Control

```
public class RemoteControlTest {  
    public static void main(String[] args) {  
        SimpleRemoteControl remote = new SimpleRemoteControl();  
        Light light = new Light();  
        LightOnCommand lightOn = new LightOnCommand(light);  
  
        remote.setCommand(lightOn);  
        remote.buttonWasPressed();  
    }  
}
```



```
public class GarageDoorOpenCommand implements Command {
```

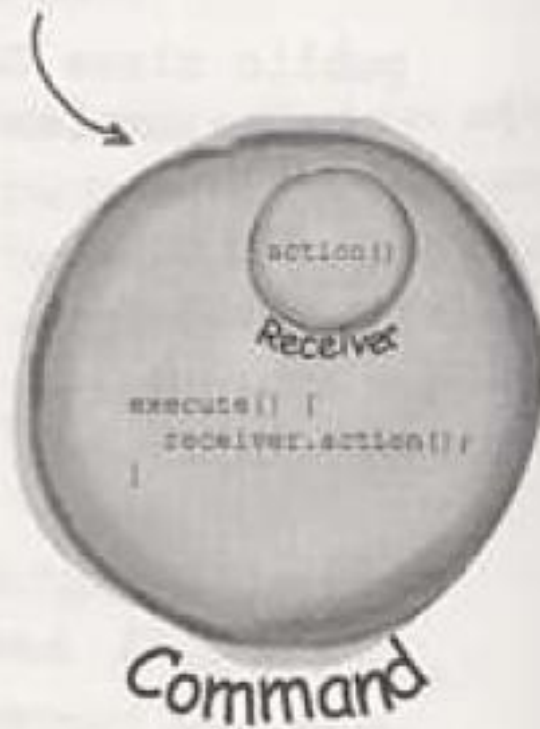


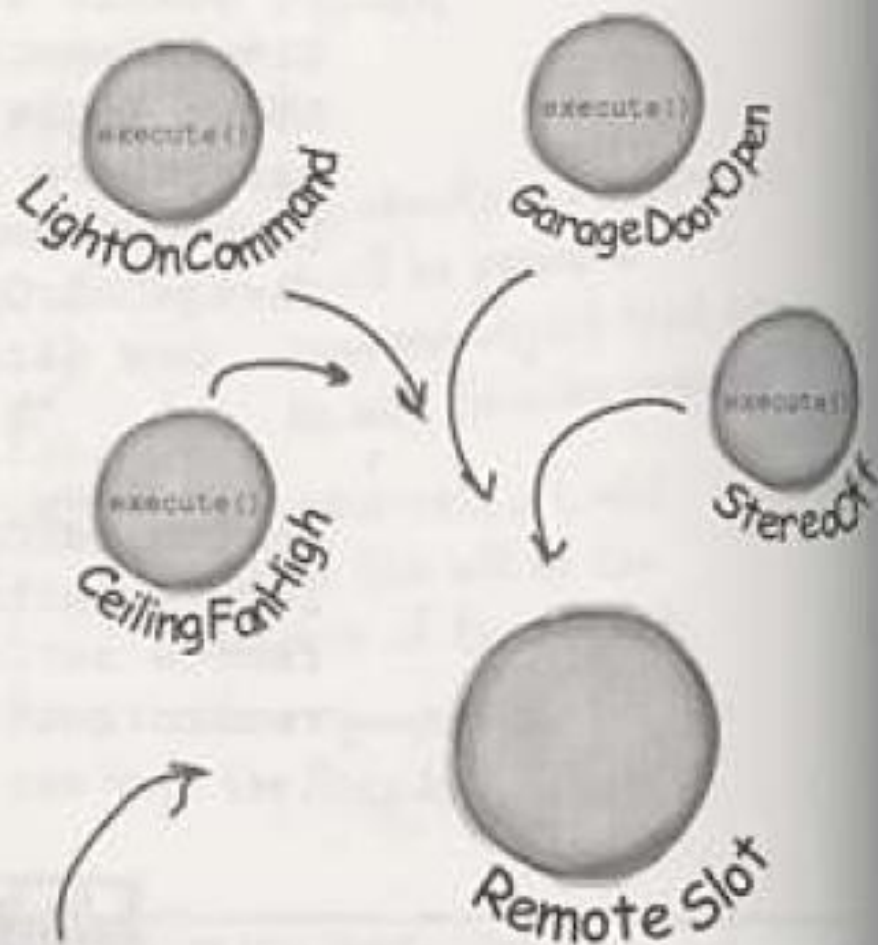
```
}
```

The Command Pattern defined

The Command Pattern encapsulates a request as an object, thereby letting you parameterize other objects with different requests, queue or log requests, and support undoable operations.

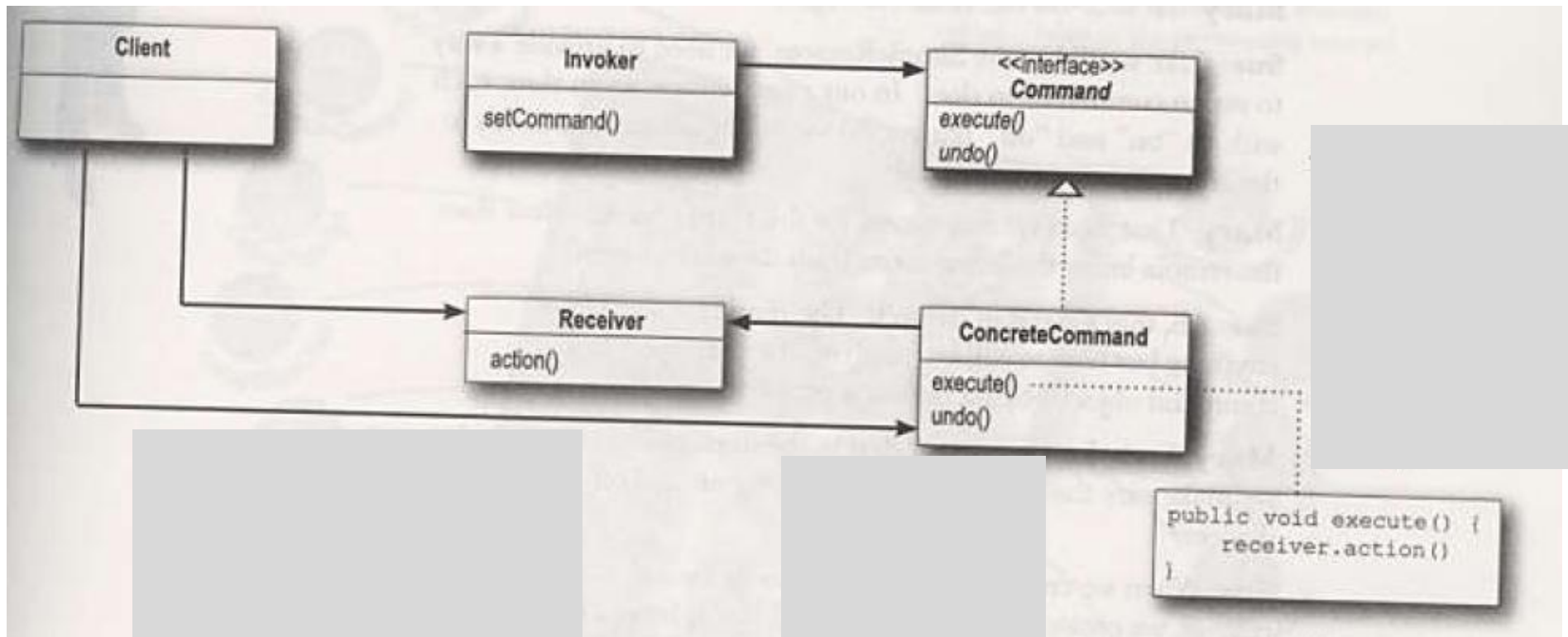
An encapsulated request





An invoker - for instance
one slot of the remote
- can be parameterized with
different requests.

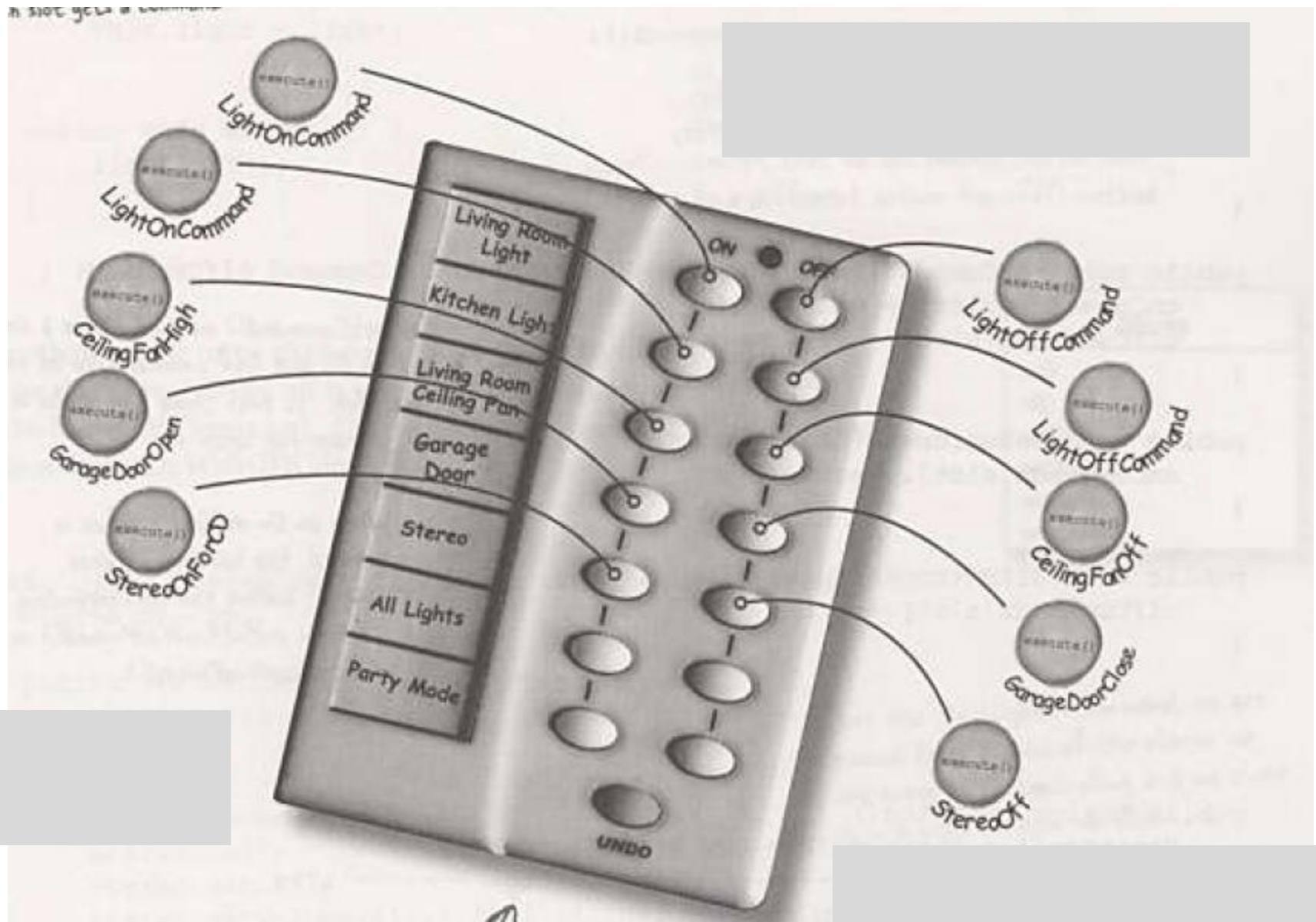
The Command Pattern defined: the class diagram





How does the design of the Command Pattern support the decoupling of the invoker of a request and the receiver of the request?

Assigning Commands to slots



Implementing the Remote Control

```
public class RemoteControl {  
    Command[] onCommands;  
    Command[] offCommands;  
  
    public RemoteControl() {  
        onCommands = new Command[7];  
        offCommands = new Command[7];  
  
        Command noCommand = new NoCommand();  
        for (int i = 0; i < 7; i++) {  
            onCommands[i] = noCommand;  
            offCommands[i] = noCommand;  
        }  
    }  
  
    public void setCommand(int slot, Command onCommand, Command offCommand) {  
        onCommands[slot] = onCommand;  
        offCommands[slot] = offCommand;  
    }  
}
```

Implementing the Remote Control

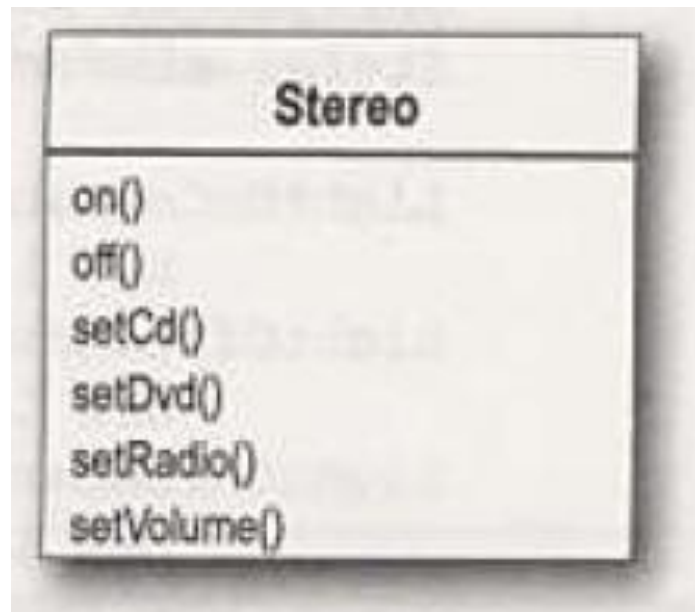
```
public void onButtonWasPushed(int slot) {
    onCommands[slot].execute();
}

public void offButtonWasPushed(int slot) {
    offCommands[slot].execute();
}

public String toString() {
    StringBuffer stringBuff = new StringBuffer();
    stringBuff.append("\n----- Remote Control ----- \n");
    for (int i = 0; i < onCommands.length; i++) {
        stringBuff.append("[slot " + i + "] " +
            onCommands[i].getClass().getName() + " " +
            offCommands[i].getClass().getName() + "\n");
    }
    return stringBuff.toString();
}
}
```

Implementing the Commands

```
public class LightOffCommand implements Command {  
    Light light;  
  
    public LightOffCommand(Light light) {  
        this.light = light;  
    }  
  
    public void execute() {  
        light.off();  
    }  
}
```



```
public class StereoOnWithCDCommand implements Command {  
    Stereo stereo;  
    public StereoOnWithCDCommand(Stereo stereo) {  
        this.stereo = stereo;  
    }  
    public void execute() {  
        stereo.on();  
        stereo.setCD();  
        stereo.setVolume(11);  
    }  
}
```

Putting the Remote Control through its paces

```
public class RemoteLoader {  
    public static void main(String[] args) {  
        RemoteControl remoteControl = new RemoteControl();  
        Light livingRoomLight = new Light("Living Room");  
        Light kitchenLight = new Light("Kitchen");  
        CeilingFan ceilingFan= new CeilingFan("Living Room");  
        GarageDoor garageDoor = new GarageDoor("");  
        Stereo stereo = new Stereo("Living Room");  
  
        LightOnCommand livingRoomLightOn =  
            new LightOnCommand(livingRoomLight);  
        LightOffCommand livingRoomLightOff =  
            new LightOffCommand(livingRoomLight);  
        LightOnCommand kitchenLightOn =  
            new LightOnCommand(kitchenLight);  
        LightOffCommand kitchenLightOff =  
            new LightOffCommand(kitchenLight);  
    }  
}
```

```
CeilingFanOnCommand ceilingFanOn =  
    new CeilingFanOnCommand(ceilingFan);  
CeilingFanOffCommand ceilingFanOff =  
    new CeilingFanOffCommand(ceilingFan);
```

```
GarageDoorUpCommand garageDoorUp =  
    new GarageDoorUpCommand(garageDoor);  
GarageDoorDownCommand garageDoorDown =  
    new GarageDoorDownCommand(garageDoor);
```

```
StereoOnWithCDCommand stereoOnWithCD =  
    new StereoOnWithCDCommand(stereo);  
StereoOffCommand stereoOff =  
    new StereoOffCommand(stereo);
```

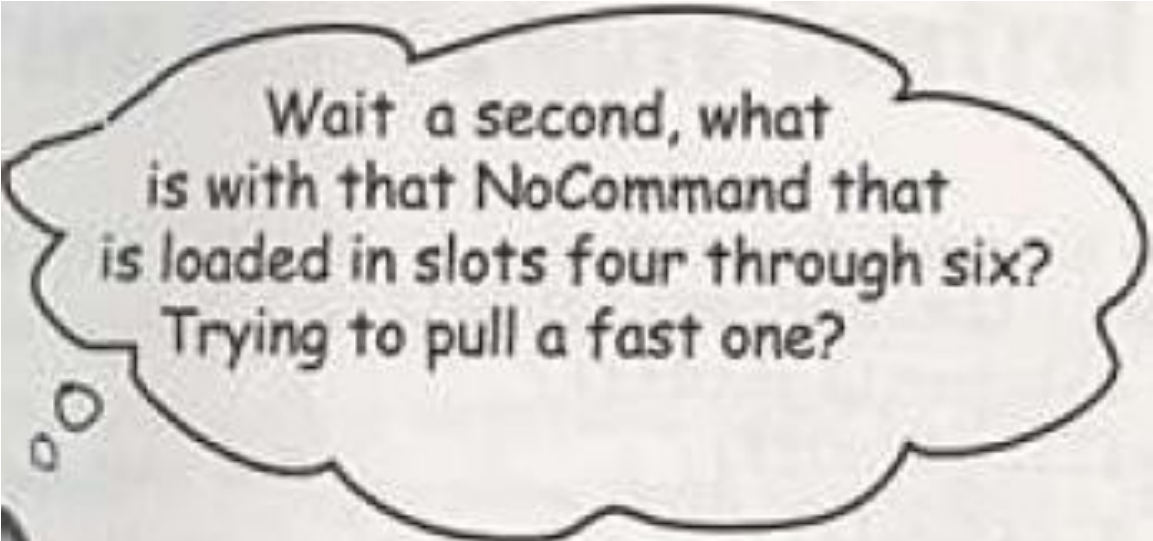
```
remoteControl.setCommand(0, livingRoomLightOn, livingRoomLightOff);  
remoteControl.setCommand(1, kitchenLightOn, kitchenLightOff);  
remoteControl.setCommand(2, ceilingFanOn, ceilingFanOff);  
remoteControl.setCommand(3, stereoOnWithCD, stereoOff);
```

```
System.out.println(remoteControl);
```

```
remoteControl.onButtonWasPushed(0);  
remoteControl.offButtonWasPushed(0);  
remoteControl.onButtonWasPushed(1);  
remoteControl.offButtonWasPushed(1);  
remoteControl.onButtonWasPushed(2);  
remoteControl.offButtonWasPushed(2);  
remoteControl.onButtonWasPushed(3);  
remoteControl.offButtonWasPushed(3);
```

```
}
```

```
}
```



Wait a second, what
is with that NoCommand that
is loaded in slots four through six?
Trying to pull a fast one?

```
public void onButtonWasPushed(int slot) {  
    if (onCommands[slot] != null) {  
        onCommands[slot].execute();  
    }  
}
```

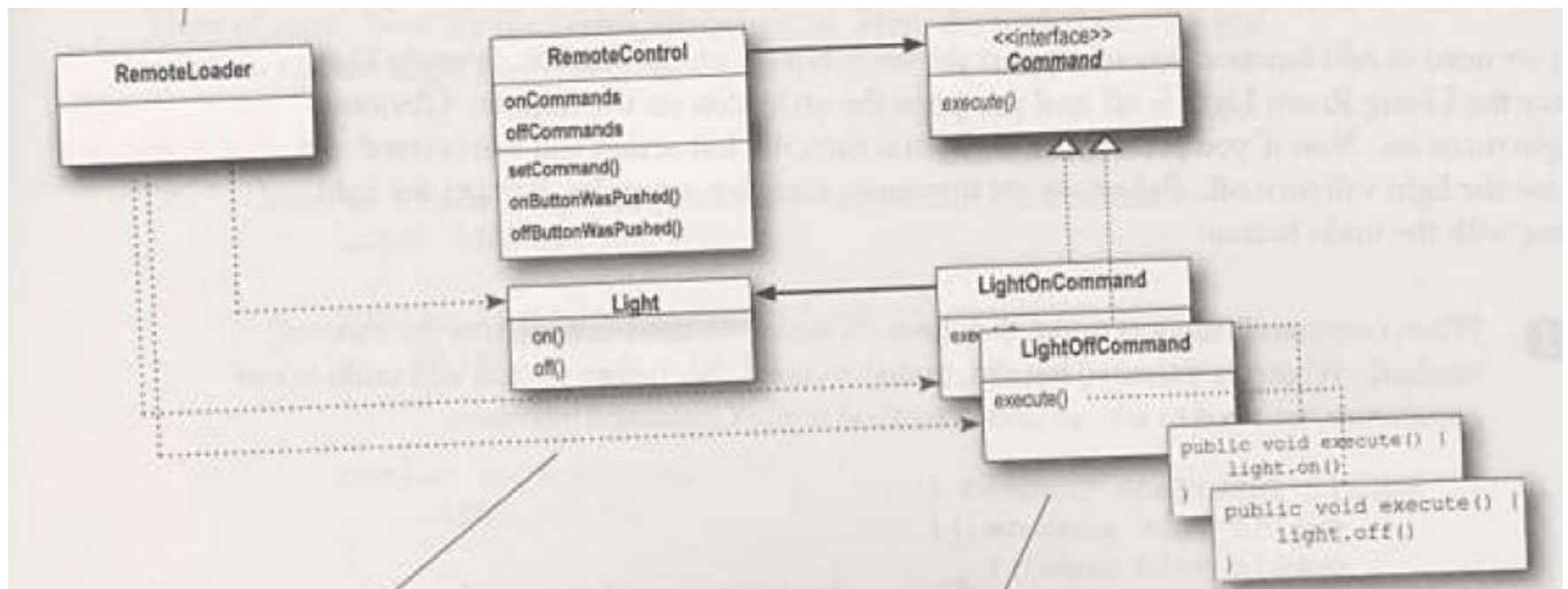
```
public class NoCommand implements Command {  
    public void execute() { }  
}
```

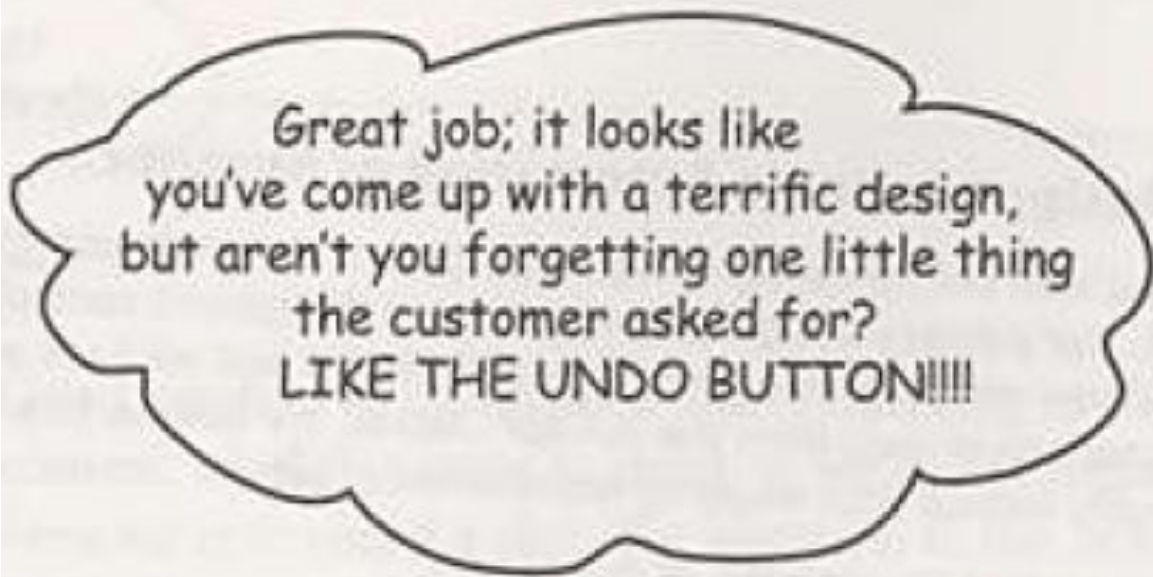
```
Command noCommand = new NoCommand();  
for (int i = 0; i < 7; i++) {  
    onCommands[i] = noCommand;  
    offCommands[i] = noCommand;  
}
```



The NoCommand object is an example of a *null object*. A null object is useful when you don't have a meaningful object to return, and yet you want to remove the responsibility for handling `null` from the client. For instance, in our remote control we didn't have a meaningful object to assign to each slot out of the box, so we provided a NoCommand object that acts as a surrogate and does nothing when its execute method is called.

You'll find uses for Null Objects in conjunction with many Design Patterns and sometimes you'll even see Null Object listed as a Design Pattern.





Great job; it looks like
you've come up with a terrific design,
but aren't you forgetting one little thing
the customer asked for?
LIKE THE UNDO BUTTON!!!!

Whoops! We almost forgot... luckily, once we
have our basic Command classes, undo is easy
to add. Let's step through adding undo to our
commands and to the remote control...

What are we doing?

```
public interface Command {  
    public void execute();  
    public void undo();  
}
```

```
public class LightOnCommand implements Command {  
    Light light;  
  
    public LightOnCommand(Light light) {  
        this.light = light;  
    }  
  
    public void execute() {  
        light.on();  
    }  
  
    public void undo() {  
        light.off();  
    }  
}
```

```
public class LightOffCommand implements Command {  
    Light light;  
  
    public LightOffCommand(Light light) {  
        this.light = light;  
    }  
  
    public void execute() {  
        light.off();  
    }  
  
    public void undo() {  
        light.on();  
    }  
}
```

```
public class RemoteControlWithUndo {
    Command[] onCommands;
    Command[] offCommands;
    Command undoCommand;

    public RemoteControlWithUndo() {
        onCommands = new Command[7];
        offCommands = new Command[7];

        Command noCommand = new NoCommand();
        for(int i=0;i<7;i++) {
            onCommands[i] = noCommand;
            offCommands[i] = noCommand;
        }
        undoCommand = noCommand;
    }

    public void setCommand(int slot, Command onCommand, Command offCommand) {
        onCommands[slot] = onCommand;
        offCommands[slot] = offCommand;
    }
}
```

```
public void onButtonWasPushed(int slot) {  
    onCommands[slot].execute();  
    undoCommand = onCommands[slot];  
}
```

```
public void offButtonWasPushed(int slot) {  
    offCommands[slot].execute();  
    undoCommand = offCommands[slot];  
}
```

```
public void undoButtonWasPushed() {  
    undoCommand.undo();  
}
```

```
public String toString() {  
    // toString code here...  
}
```

```
}
```

Using state to implement Undo

```
public class CeilingFan {  
    public static final int HIGH = 3;  
    public static final int MEDIUM = 2;  
    public static final int LOW = 1;  
    public static final int OFF = 0;  
    String location;  
    int speed;  
  
    public CeilingFan(String location) {  
        this.location = location;  
        speed = OFF;  
    }  
  
    public void high() {  
        speed = HIGH;  
        System.out.println(location + " ceiling fan is on high");  
    }  
}
```

```
public void medium() {  
    speed = MEDIUM;  
    System.out.println(location + " ceiling fan is on medium");  
}
```

```
public void low() {  
    speed = LOW;  
    System.out.println(location + " ceiling fan is on low");  
}
```

```
public void off() {  
    speed = OFF;  
    System.out.println(location + " ceiling fan is off");  
}
```

```
public int getSpeed() {  
    return speed;  
}
```

```
}
```

Adding Undo to the ceiling fan commands

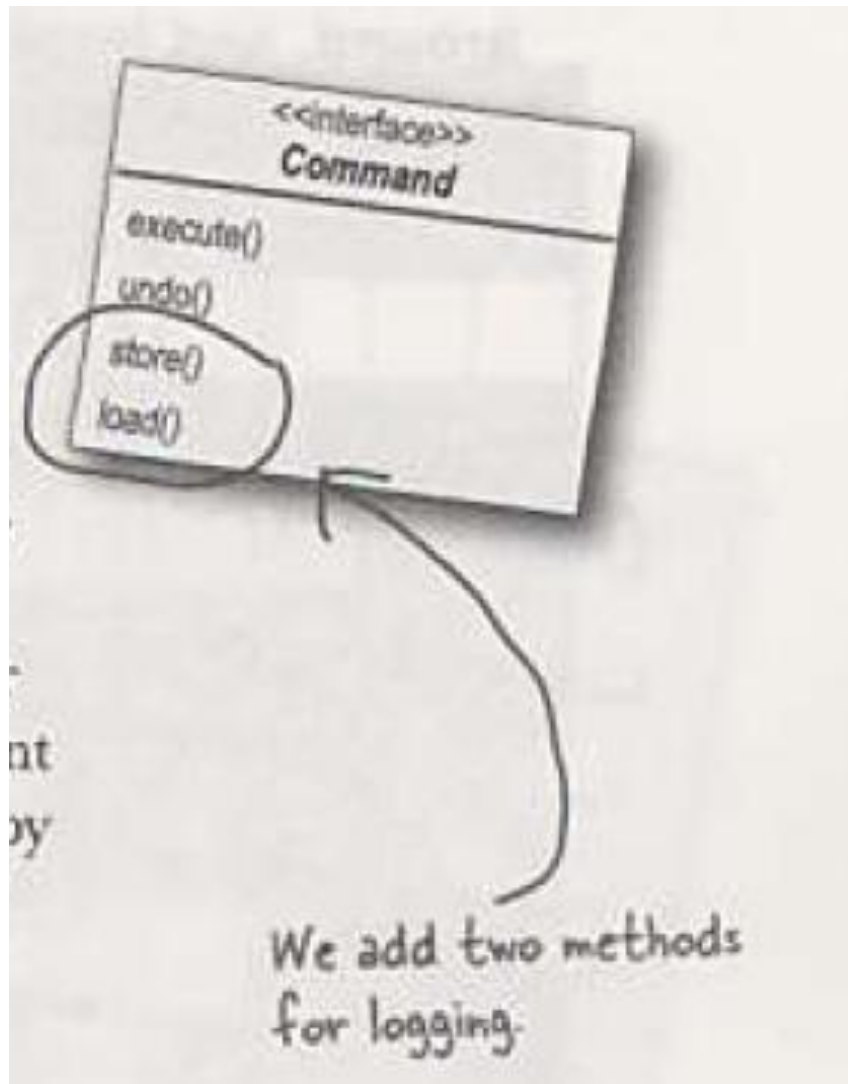
```
public class CeilingFanHighCommand implements Command {
    CeilingFan ceilingFan;
    int prevSpeed;
    public CeilingFanHighCommand(CeilingFan ceilingFan) {
        this.ceilingFan = ceilingFan;
    }
    public void execute() {
        prevSpeed = ceilingFan.getSpeed();
        ceilingFan.high();
    }
    public void undo() {
        if (prevSpeed == CeilingFan.HIGH) {
            ceilingFan.high();
        } else if (prevSpeed == CeilingFan.MEDIUM) {
            ceilingFan.medium();
        } else if (prevSpeed == CeilingFan.LOW) {
            ceilingFan.low();
        } else if (prevSpeed == CeilingFan.OFF) {
            ceilingFan.off();
        }
    }
}
```

More uses of the Command Pattern: queuing requests

Objects implementing the command interface are added to the queue.

Threads remove commands from the queue one by one and call their `execute()` method. Once complete, they go back for a new command object.

More uses of the Command Pattern: logging requests



As each command is executed, it is stored on disk.



After a system failure, the objects are reloaded and executed in the correct order.

Homework

- Façade Pattern
- Structural Pattens

References

- Head First Design Patterns by Eric, Gamma
- <https://refactoring.guru/design-patterns/command>
- <https://refactoring.guru/design-patterns/facade>